llnl_c1/lulesh · lulesh.cc · llm_pragmas_v5 · claude-haiku-4-5-20251001 · BROKEN

−73 / +267 lines of 1967 · 65 parallel region(s) added · 30 of 91 candidate(s) kept · RESTRUCTURING (155 code line(s) added, 71 removed) · restructuring level 0

BEFORE

```
109  void InitStressTermsForElems(Domain &domain,
110                               Real_t *sigxx, Real_t *sigyy, Real_t *sigzz,
111                               Index_t numElem)
112  {
113
114     for (Index_t i = 0 ; i < numElem ; ++i){
115        sigxx[i] = sigyy[i] = sigzz[i] =  - domain.p(i) - domain.q(i) ;
116     }
117  }
118
     …
308
309     Real_t fx_local[8] ;
310     Real_t fy_local[8] ;
311     Real_t fz_local[8] ;
312
313     for( Index_t k=0 ; k<numElem ; ++k )
314     {
315       const Index_t* const elemToNode = domain.nodelist(k);
316       Real_t B[3][8] ;
317       Real_t x_local[8] ;
     …
329       SumElemStressesToNodeForces( B, sigxx[k], sigyy[k], sigzz[k],
330                                    fx_local, fy_local, fz_local ) ;
331
332       for( Index_t lnode=0 ; lnode<8 ; ++lnode ) {
333          Index_t gnode = elemToNode[lnode];
334          domain.fx(gnode) += fx_local[lnode];
335          domain.fy(gnode) += fy_local[lnode];
336          domain.fz(gnode) += fz_local[lnode];
337       }
338
339     }
340
341  }
342
343  static inline
344  void VoluDer(const Real_t x0, const Real_t x1, const Real_t x2,
     …
495     gamma[3][4] = Real_t( 1.);
496     gamma[3][5] = Real_t(-1.);
497     gamma[3][6] = Real_t( 1.);
498     gamma[3][7] = Real_t(-1.);
499
500     for(Index_t i2=0;i2<numElem;++i2){
501        Real_t hgfx[8], hgfy[8], hgfz[8] ;
502
503        Real_t coefficient;
504
     …
607
608        CalcElemFBHourglassForce(xd1,yd1,zd1,
609                                 hourgam,
```

AFTER

```
109  void InitStressTermsForElems(Domain &domain,
110                               Real_t *sigxx, Real_t *sigyy, Real_t *sigzz,
111                               Index_t numElem)
112  {
113
114     #pragma omp parallel for schedule(static)
115     for (Index_t i = 0 ; i < numElem ; ++i){
116        sigxx[i] = sigyy[i] = sigzz[i] =  - domain.p(i) - domain.q(i) ;
117     }
118  }
119
     …
309
310     Real_t fx_local[8] ;
311     Real_t fy_local[8] ;
312     Real_t fz_local[8] ;
313
314     /* Allocate temporary buffers to accumulate stress forces per node.
315        Different elements can share nodes, so direct parallel accumulation
316        into domain.fx/fy/fz would cause races. These buffers accumulate safely
317        with atomic operations, then are merged sequentially into domain. */
318     Real_t *fx_temp = Allocate<Real_t>(numNode);
319     Real_t *fy_temp = Allocate<Real_t>(numNode);
320     Real_t *fz_temp = Allocate<Real_t>(numNode);
321
322     memset(fx_temp, 0, numNode * sizeof(Real_t));
323     memset(fy_temp, 0, numNode * sizeof(Real_t));
324     memset(fz_temp, 0, numNode * sizeof(Real_t));
325
326     #pragma omp parallel for schedule(static) private(fx_local, fy_local, fz_loc
327     for( Index_t k=0 ; k<numElem ; ++k )
328     {
329       const Index_t* const elemToNode = domain.nodelist(k);
330       Real_t B[3][8] ;
331       Real_t x_local[8] ;
     …
343       SumElemStressesToNodeForces( B, sigxx[k], sigyy[k], sigzz[k],
344                                    fx_local, fy_local, fz_local ) ;
345
346       for( Index_t lnode=0 ; lnode<8 ; ++lnode ) {
347          Index_t gnode = elemToNode[lnode];
348          #pragma omp atomic
349          fx_temp[gnode] += fx_local[lnode];
350          #pragma omp atomic
351          fy_temp[gnode] += fy_local[lnode];
352          #pragma omp atomic
353          fz_temp[gnode] += fz_local[lnode];
354       }
355
356     }
357
358     /* Merge accumulated stress forces from temporary buffers into domain.
359        This sequential merge is safe and the bulk of computation was paralleliz
360     for(Index_t i=0; i<numNode; ++i) {
361        domain.fx(i) += fx_temp[i];
362        domain.fy(i) += fy_temp[i];
363        domain.fz(i) += fz_temp[i];
364     }
365
366     Release(&fx_temp);
367     Release(&fy_temp);
368     Release(&fz_temp);
369
370  }
371
372  static inline
373  void VoluDer(const Real_t x0, const Real_t x1, const Real_t x2,
     …
524     gamma[3][4] = Real_t( 1.);
525     gamma[3][5] = Real_t(-1.);
526     gamma[3][6] = Real_t( 1.);
527     gamma[3][7] = Real_t(-1.);
528
529     /* Allocate temporary buffers to accumulate forces per node.
530        Different elements can share nodes, so direct parallel accumulation
531        into domain.fx/fy/fz would cause races. These buffers are accumulated
532        into sequentially after the parallel loop. */
533     Real_t *fx_temp = Allocate<Real_t>(numNode);
534     Real_t *fy_temp = Allocate<Real_t>(numNode);
535     Real_t *fz_temp = Allocate<Real_t>(numNode);
536
537     memset(fx_temp, 0, numNode * sizeof(Real_t));
538     memset(fy_temp, 0, numNode * sizeof(Real_t));
539     memset(fz_temp, 0, numNode * sizeof(Real_t));
540
541     #pragma omp parallel for schedule(static) shared(gamma, numElem, numNode, d
542     for(Index_t i2=0;i2<numElem;++i2){
543        Real_t hgfx[8], hgfy[8], hgfz[8] ;
544
545        Real_t coefficient;
546
     …
649
650        CalcElemFBHourglassForce(xd1,yd1,zd1,
651                                 hourgam,
```

file

red = removed green = added yellow = parallel region (#pragma omp) right strip = the whole file: green where changed, yellow where a parallel region now is